

RadarRun Pipeline AS IS

Current as of Slice 2.17.4.6.1.3.4.0.1.

This document is the factual runtime contract for the current RadarRun pipeline. It describes what the product does today, what evidence proves it, and which boundaries must be protected when future search, signal, and candidate slices change the system.

The broader upstream architecture map stays in `docs/architecture/UPSTREAM_SEARCH_AND_SIGNAL_ARCHITECTURE.md`. This file is the dedicated RadarRun pipeline AS IS source, symmetric to `DRAFT_RUN_PIPELINE_AS_IS.md`.

PDF quick view: `docs/architecture/RADAR_RUN_PIPELINE_AS_IS.pdf`.

Regenerate with:

```
python scripts/generate-draft-run-pipeline-pdf.py --source docs/architecture/RADAR_RUN_PIPELINE_AS_IS.md --output docs/archi
```

How AS IS Participates in DoD

For any complex RadarRun, upstream search, source signal, scoring, benchmark, or candidate assembly slice, this document must be used twice:

1. Before implementation, as a requirement source for the slice DoD.
2. After implementation, as a validator for what changed or stayed invariant.

A DoD for a RadarRun slice must explicitly state:

- which AS IS invariants are preserved;
- which AS IS behavior is intentionally changed;
- which TO BE or ADR document authorizes the change;
- which runtime artifacts prove the result;
- whether this document and PDF must be updated after the slice.

Required DoD evidence for RadarRun/search work comes from structured artifacts, not from subjective inspection:

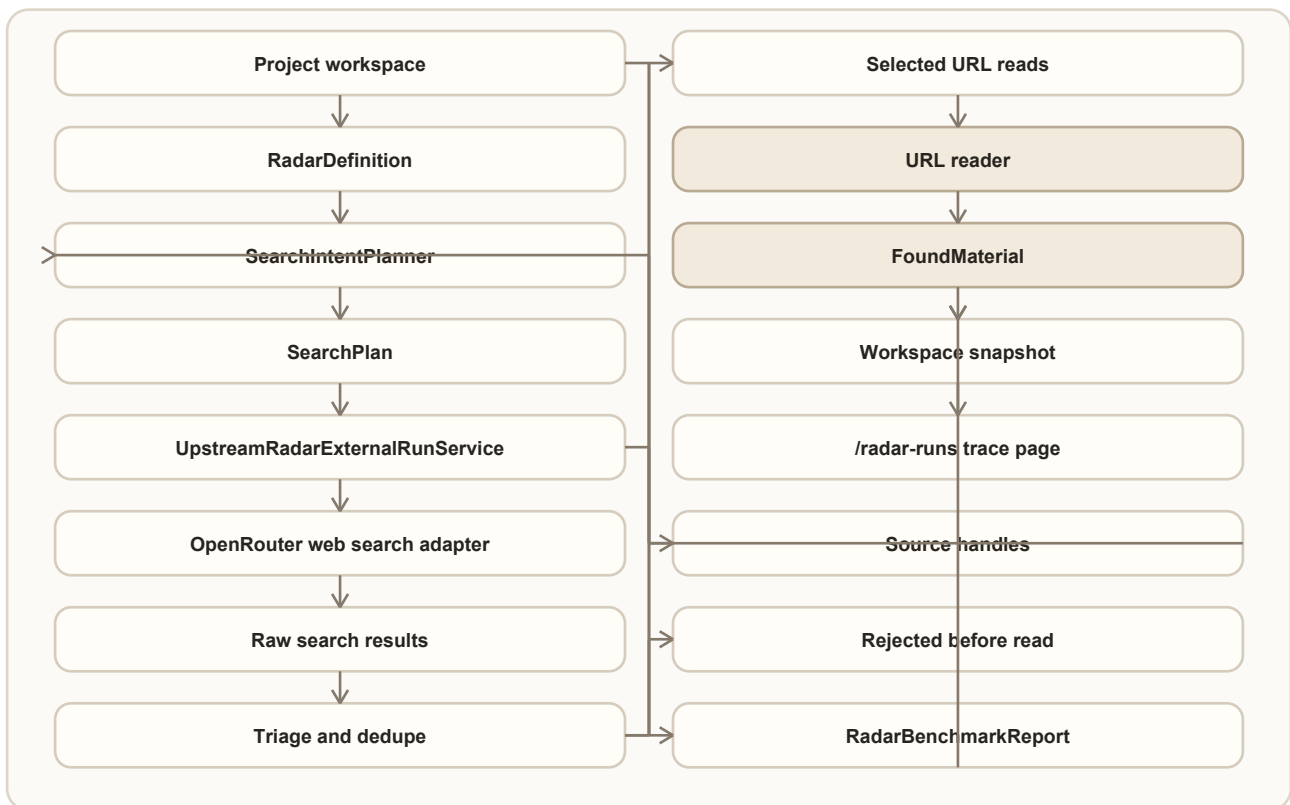
- `searchPlan.strategy`, `language`, `intents`, `queries`, `sourceStrategy`, `skippedIntents`, and `skippedIntentDetails`;
- planned and executed coverage: `plannedCoverage`, `executedCoverage`, and `skippedRequiredCoverage`;
- operation timeline: provider query operations, skipped operations, warnings, and errors;
- `rawResults`, `selectedForRead`, `rejectedBeforeRead`, URL-read outcomes, and `foundMaterialIds`;
- `FoundMaterial` records stored in the project workspace;
- `benchmarkReport` when a matching scenario exists;
- `/radar-runs?runId=<id>` trace page rendering for human diagnostics.

If a slice changes RadarRun behavior but does not update this AS IS contract or state why the contract remained valid, the slice is not ready to close.

Core Concepts

Concept	Current role	Does not own
SourceHandle	Project-scoped descriptor for a source that may be searchable, readable-only, paused, or needs review.	Provider execution, signal approval, candidate ranking.
RadarDefinition	User-facing radar settings: scope, source handles, trigger rules, filters, execution mode, and budget caps.	Search result storage or downstream drafting.
SearchPlan	Deterministic provider-free campaign plan with strategy, language, intents, queries, source strategy, budget, and skipped intents.	Provider calls or quality scoring.
SearchIntent	One planned evidence direction such as broad discovery, case/example, benchmark/paper, OSS/tooling, limitation/critique, or freshness.	URL reading or material acceptance.
SearchQuery	One provider-executable web-search query derived from an intent and eligible source strategy.	Search result quality judgment.
RadarRun	One execution attempt: status, budget usage, operations, search plan, raw results, read decisions, material ids, warnings, errors, and optional benchmark report.	SourceSignal, PostCandidate, plan slot, or DraftRun creation.
RadarRunOperation	One provider/search/read operation with status and trace-safe payload.	Editorial approval.
RawSearchResult	Normalized provider search result with query/run provenance.	Durable source memory or signal ownership.
selectedForRead	Search results chosen for URL reading within budget.	Proof that the material is accepted.
rejectedBeforeRead	Results rejected before URL reading, including duplicates, budget skips, and noise.	Permanent deletion from future search memory.
FoundMaterial	Retrieval output with provenance, title, URL/source ref, snippet or summary, warnings, and captured timestamp.	Topic/fabula ownership or final post candidate approval.
RadarBenchmarkReport	Recorded or live evaluation against a golden scenario.	Search execution or UI-side scoring.
RadarRunTracePage	Frontend read model for inspecting one run.	Recomputing live quality or mutating the run.

Runtime Topology



The current implementation stores RadarRun and FoundMaterial data in the workspace snapshot. There is no dedicated RadarRun SQLite table and no separate HTTP trace endpoint in this AS IS state.

Current Step Order

1. Resolve project, workspace, radar, source handles, language, research depth, and execution mode.
2. Classify source handles by eligibility:
 - searchable;
 - readable-only;
 - paused;
 - needs review;
 - unavailable or unsupported.
3. Build a deterministic SearchPlan:
 - source strategy;
 - campaign trace;
 - planned intents;
 - query families;
 - evidence types;
 - budget caps;
 - skipped intents and reasons.
4. Apply maxExternalQueries and produce provider-executable queries[].
5. Run provider web-search operations for executable queries.
6. Normalize provider citations into rawResults[] with queryId provenance.

7. Deduplicate and triage raw results.
8. Select a bounded set of URL reads.
9. Record rejected-before-read results and reasons.
10. Read selected URLs when URL reading is available.
11. Create `FoundMaterial` for usable reads or search-result-only fallbacks.
12. Attach `benchmarkReport` when the run matches a golden scenario.
13. Persist the updated workspace snapshot.
14. Render compact radar trace and, when opened, the dedicated `/radar-runs` trace page.

Context Handoff and Execution Contract

RadarRun does not pass hidden state between roles. The handoff is persisted in the workspace snapshot and in the run payload.

Stage	Input artifacts	Output artifacts	Required proof
Source eligibility	Project workspace, RadarDefinition, source handles	<code>searchPlan.sourceStrategy</code> , skipped source reasons	Trace shows searchable, readable-only, paused, and needs-review handles.
Campaign planning	Radar scope, language, topics/fabulas as context, publisher/editorial rules, source strategy, budget mode	<code>searchPlan.intents</code> , queries, <code>skippedIntentDetails</code> , campaign trace	Planned intents and skipped reasons are visible in <code>searchPlan</code> .
Query budgeting	Planned intents, <code>maxExternalQueries</code>	Bounded <code>queries[]</code> , skipped intent reasons	Required directions skipped by budget appear as skipped coverage.
Provider search	Executable <code>queries[]</code> , provider config	<code>RadarRunOperation</code> , provider citations, raw results	Operation status, query id, provider errors, warnings, and raw result provenance.
Triage and dedupe	<code>rawResults[]</code> , read budget, duplicate URLs, noise rules	<code>selectedForRead</code> , <code>rejectedBeforeRead</code>	Selected and rejected counts plus per-result reasons.
URL read	Selected reads, URL reader adapter	read payloads, warnings, search-result-only fallback where available	URL-read operation status and material warnings.
Material output	Search/read payloads	<code>FoundMaterial</code> , <code>foundMaterialIds</code>	Workspace contains the material and run links it by id.
Benchmark evaluation	Scenario, run, found materials	<code>benchmarkReport</code>	Recorded/live status, provider health, coverage, missing expectations, and noise hits.

Hard Output Boundaries

RadarRun is a retrieval and trace pipeline. It may create `FoundMaterial`.

It must not create:

- `SourceSignal`;
- `PostCandidate`;
- plan slots;
- `DraftRun`;
- final topic/fabula ownership for raw material.

Those downstream artifacts belong to separate review, scoring, candidate assembly, and planning slices. A RadarRun may expose affinity context, but it must not silently turn raw search results into approved editorial work.

Trace Contract

The current RadarRun trace contract is intentionally explicit. Future work must not collapse it into a single "results" blob.

Trace field	Meaning	Why it matters
<code>searchPlan.strategy</code>	Campaign strategy chosen by the deterministic planner.	Explains why this run searched in this shape.
<code>searchPlan.language</code>	Query language.	Prevents hidden RU/EN drift.
<code>searchPlan.intents[]</code>	Planned evidence directions.	Proves what the radar wanted to cover.
<code>searchPlan.queries[]</code>	Provider-executable queries.	Proves what could actually run under budget.
<code>searchPlan.sourceStrategy</code>	Source handle eligibility and use.	Explains searchable versus readable-only sources.
<code>searchPlan.skippedIntents[]</code> and <code>skippedIntentDetails[]</code>	Directions not executed and why.	Makes budget and source gaps visible.
<code>operations[]</code>	Provider and read operations.	Separates provider/runtime health from quality.
<code>rawResults[]</code>	Normalized provider citations.	Shows what the provider returned before triage.
<code>selectedForRead[]</code>	Raw results chosen for URL reading.	Shows read-budget choices.
<code>rejectedBeforeRead[]</code>	Raw results rejected before read.	Shows duplicates, noise, and budget skips.
<code>foundMaterialIds[]</code>	Materials created by this run.	Links run trace to stored source material.
<code>warnings[]</code> and <code>errors[]</code>	Runtime and quality warnings/errors.	Prevents silent degradation.
<code>benchmarkReport</code>	Golden scenario verdict when available.	Gives a stable quality signal for matching runs.

The `/radar-runs?runId=<id>` page is a read-only view over this trace. It does not compute live quality in React.

Benchmark and Live Evaluation Contract

There are two benchmark modes:

- `recorded`: deterministic fixture-backed regression. It does not call providers.
- `live`: provider-backed run evaluated against golden expectations.

The live report must distinguish planned coverage from executed coverage:

- `plannedCoverage`: directions present in `searchPlan.intents[]`;
- `executedCoverage`: directions that produced executable queries, successful search operations, raw results, selected reads, or found materials;
- `skippedRequiredCoverage`: required directions that were planned but skipped or not actually executed, including reasons such as `budget-max-external-queries`.

Status vocabulary:

Status	Meaning
<code>passed</code>	Provider was usable, required coverage was actually executed, trace is complete, enough material/domain diversity exists, and known noise was not accepted.

Status	Meaning
warning	Provider was usable and useful material exists, but there are non-blocking gaps such as required planned directions skipped by budget, optional family gaps, provider degradation, or narrow read coverage.
failed	Provider was usable, but quality failed: required coverage/material/domain checks missed or accepted material contains known noise.
inconclusive	Provider/runtime state prevents a fair quality verdict: provider disabled, missing configuration, rate limit, network failure, or no honest execution trace.

passed means the search actually covered enough of the golden scenario. It does not mean only that the plan looked good.

Reading a RadarRun Trace

Use:

```
/radar-runs?runId=<RadarRun ID>
```

The trace should be read in this order:

1. Summary cards: status, budget, source coverage, material output, warnings/errors.
2. Campaign plan: strategy, language, source strategy, intent families, skipped intents.
3. Operations: provider query and URL-read statuses.
4. Raw results: provider citations before triage.
5. Selected and rejected reads: what the run read and what it skipped.
6. Found materials: what became durable upstream material.
7. Benchmark report: whether the golden scenario verdict is passed, warning, failed, or inconclusive.
8. Raw JSON fallback when a legacy/minimal run lacks richer fields.

Known AS IS Limitations

These are current facts, not target architecture:

- Golden evaluation currently has one primary industrial AI scenario. More scenarios are needed before it becomes a broad quality gate.
- Search results are not yet stored in a reusable cross-run search memory. Rejected or unread results may be useful later, but current logic does not own that cache.
- The live query budget can skip required families such as limitation/critique. The report exposes this as warning-class coverage debt; the planner/budget tuning is a future product-quality slice.
- URL-read budget is intentionally narrow. A run may produce many raw results but read only a small subset.
- Query family wording can still be too similar across broad discovery, case/example, and benchmark/paper directions.

- Signal extraction, signal scoring, and candidate assembly remain downstream and are not owned by RadarRun.
- The frontend trace page can display historical/minimal runs, but older runs may not contain all enriched fields.

Known limitations must be linked to roadmap slices rather than treated as successful target behavior.

Maintenance Rules

- Keep this document factual. Put future designs in TO BE documents or ADRs.
- If a RadarRun slice changes runtime order, trace shape, benchmark semantics, or output boundaries, update this Markdown and regenerate the PDF in the same slice.
- If a slice only changes code organization without changing runtime behavior, state that this AS IS remains valid in the final report.
- Do not use this document to bless hidden state, unbounded provider payloads, or downstream artifact creation inside RadarRun.